



UNIVERSITÀ DI PARMA

Image Interpolation

- What is image interpolation, and why do we need it
- 1D interpolation: zero-order, first-order (linear), third-order (cubic)
- 2D interpolation as two sequential 1D passes
- Limitations of linear methods and edge-aware interpolation
- Applications: digital zooming, inpainting, geometric transformations, demosaicing

- R. Szeliski, "Computer Vision: Algorithms and Applications", Chapter 3 (Image Processing)
- G. Menegaz, "Image Interpolation" teaching slides (base material for this lecture)
- Z. Li, M. Orchard, "New Edge-Directed Interpolation", IEEE Trans. Image Processing, 2001
- OpenCV documentation: `cv::resize`, `cv::remap`

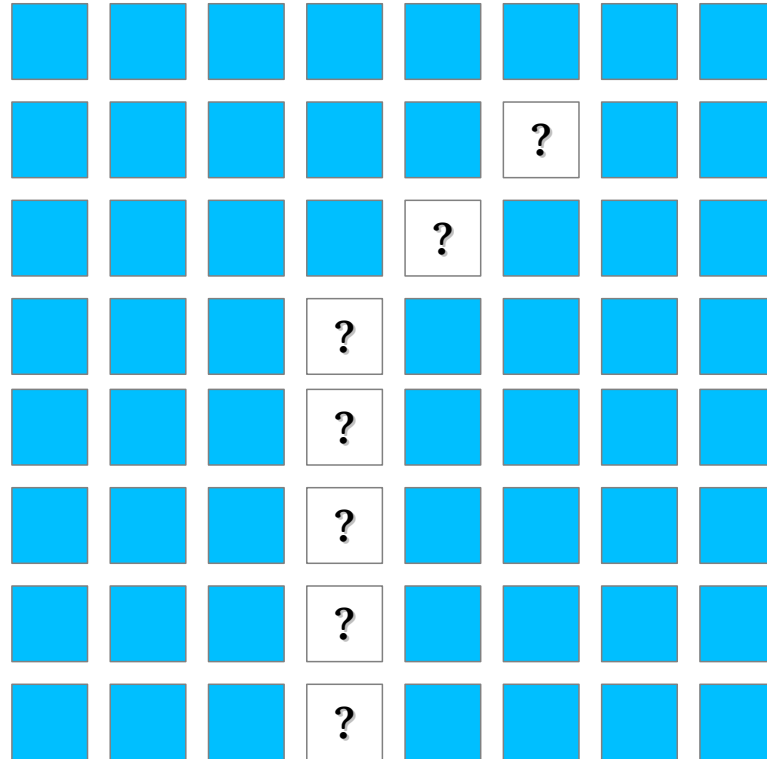
What Is Image Interpolation?

- An image $f(x,y)$ gives intensity values only at integer lattice locations (x, y integers)
- Interpolation estimates the intensity at arbitrary, non-integer locations
 - Sub-pixel precision
- It is fundamentally a "guess": every real sensor has a finite sampling pitch
- Formally: reconstruct a continuous function from a discrete set of samples, then resample it wherever needed

Why Do We Need It? Engineering Motivations

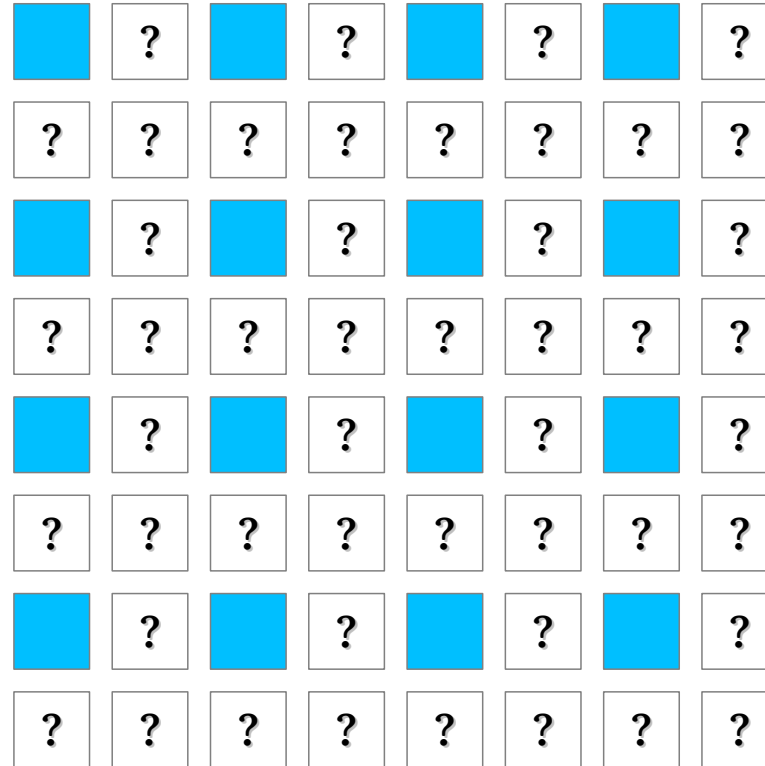
- We want BIG images:
 - full-screen playback of low-resolution video
 - digital zoom on a photo
- We want GOOD images:
 - repair a block of pixels lost or corrupted during transmission
- We want COOL images:
 - apply artistic geometric effects, warps, morphing

Image inpainting



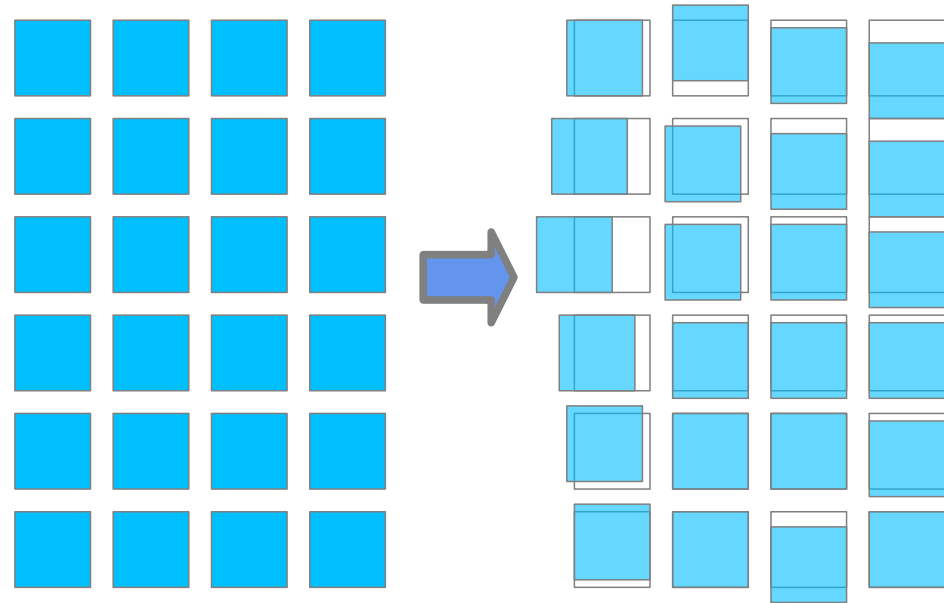
No sub-pixel
issues

Image resize



Potentially
sub-pixel
issues

Image warping/morphing



Sub-pixel issues

Recall: How a Digital Image Is Formed

- An image is a discrete point-sampling of a continuous function of the scene radiance
- If we could reconstruct that original continuous function, we could regenerate the image at any resolution or scale
- This is exactly the sampling/reconstruction problem you should already met (Nyquist)
- Interpolation = practical, approximate reconstruction + resampling

From Samples to a Continuous Function

- Convert the discrete samples $f[n]$ to a continuous function via convolution with a reconstruction filter h
- Ideal reconstruction filter:
 - a square window (ideal low-pass) in the frequency domain
 - a sinc function in the spatial domain
 - theoretically perfect, but impractical: infinite support, ringing/truncation artifacts, high computational cost
- In practice we use simpler, finite-support filters: nearest-neighbor, linear, Gaussian, cubic
- Trade-off: smoother/sharper reconstruction vs. computational cost and artifacts

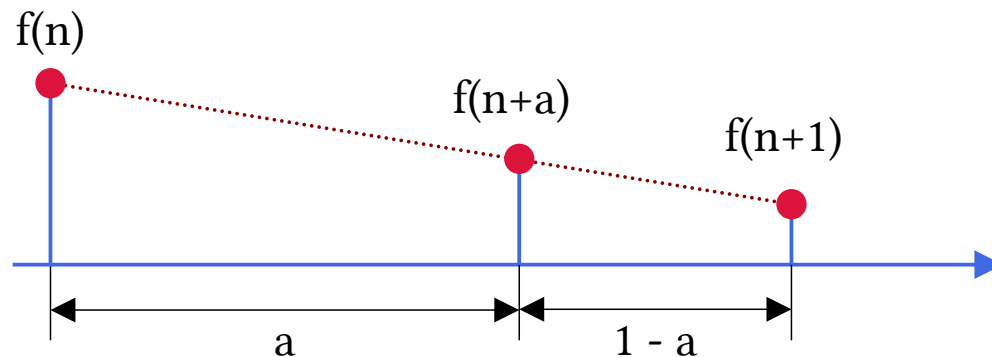
Zero-order Interpolation (Replication)

- Each new sample takes the value of the closest known sample
- Simplest possible upsampling: repeat each row and column
- Pros / cons:
 - very fast, no new intensity values are invented
 - produces blocky, jagged results, especially at edges



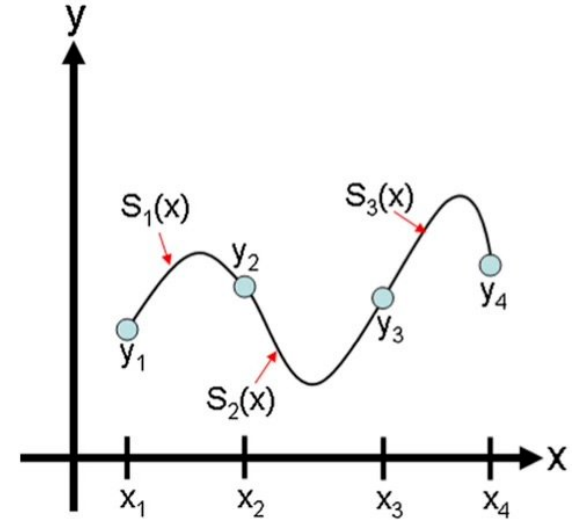
First-order Interpolation (Linear)

- Heuristic: the closer we are to a known sample, the more weight it gets
- $f(n+a) = (1 - a) \cdot f(n) + a \cdot f(n+1)$
 - with $0 < a < 1$
- When $a = 0.5$, this is simply the average of the two neighboring samples
- Geometric interpretation: fitting a straight line (first-degree polynomial) between neighboring samples



Third-order Interpolation (Cubic)

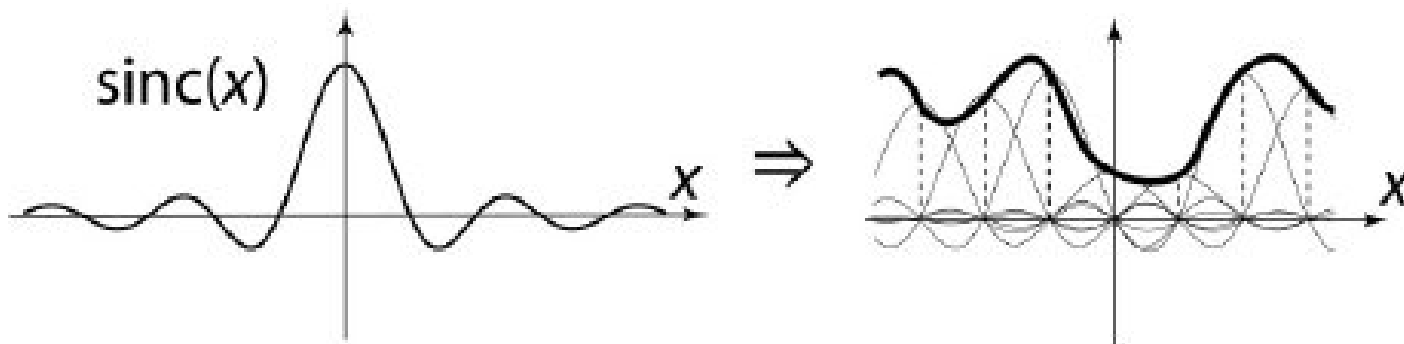
- Uses four neighboring samples instead of two, fitting a cubic polynomial (cubic spline)
- Produces smoother results than linear interpolation, better preserving curvature and gradients
- Basis of "bicubic" interpolation in 2D (e.g. `cv::INTER_CUBIC`)
- Higher quality, but higher computational cost than zero- or first-order methods



- Nyquist-Shannon theorem
 - Sampling with $f_s \geq 2f_{\max} \rightarrow$ complete reconstruction
 - We can not act on $f_s \rightarrow$ ideal reconstruction
- From a frequency point of view $\rightarrow$ “brick wall” pass filter with $f_s/2$ “size”
- In the spatial domain $\rightarrow$ a sinc function

$$f(x) = \sum_n f(n) \cdot \text{sinc}(x - n)$$

- Practical limits
 - Infinite span
 - Large number of samples to use
 - Expensive



- 2D interpolation = two sequential 1D interpolations (divide and conquer)
- First interpolate along rows, then along columns (or vice versa)
— same idea as separable convolution kernels
- Bilinear interpolation formula:

$$h(x.a, y.b) = (1-a)(1-b) \cdot f(x, y) + (1-a)b \cdot f(x, y+1) + a(1-b) \cdot f(x+1, y) + ab \cdot f(x+1, y+1)$$

$$a, b \in [0, 1]$$

Worked Example: Zero- vs First-order in 2D

- Resize example 2x
- Given four known samples arranged as a 2×2 block: a, b (top row), c, d (bottom row)
- Zero-order (replication), each sample repeated in a 4×4 block:
 - a a b b / a a b b / c c d d / c c d d
- First-order (bilinear), new values are weighted averages:
 - corners stay a, b, c, d
 - edge midpoints: $(a+b)/2$, $(a+c)/2$, $(b+d)/2$, $(c+d)/2$
 - center: $(a+b+c+d)/4$

a	b
c	d

Visual Comparison: NN vs Bilinear vs Bicubic

- Qualitative trade-offs when upsampling the same low-resolution image:
 - Nearest-neighbor: fastest, visibly blocky
 - Bilinear: smoother, but can blur edges and fine detail
 - Bicubic: sharper than bilinear, better detail preservation, higher cost
- General rule: quality increases ($\text{NN} < \text{bilinear} < \text{bicubic}$) as computational cost increases

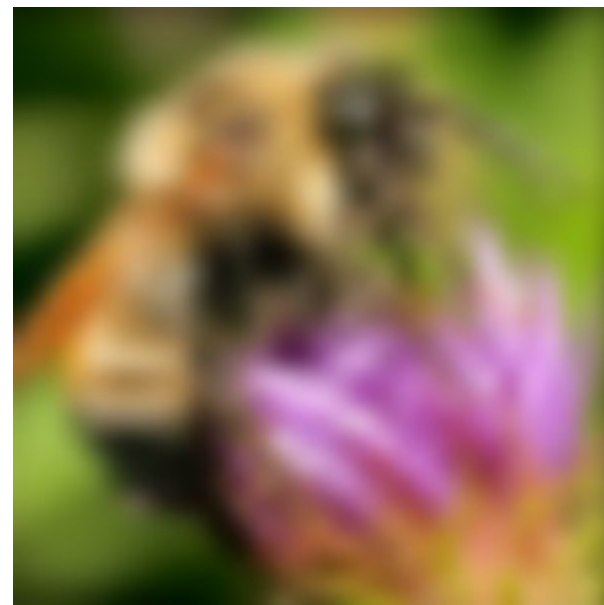
Visual Comparison



Nearest-neighbor interpolation



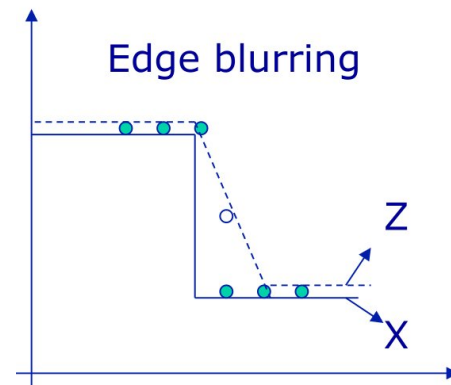
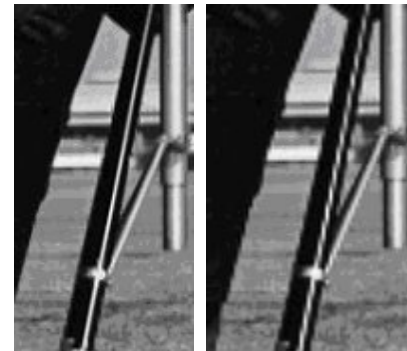
Bilinear interpolation



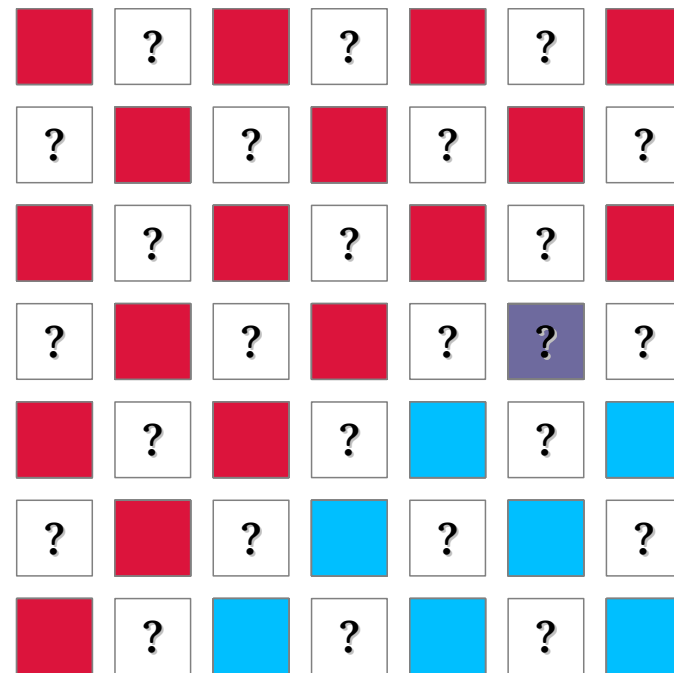
Bicubic interpolation

Limitations of Linear Methods

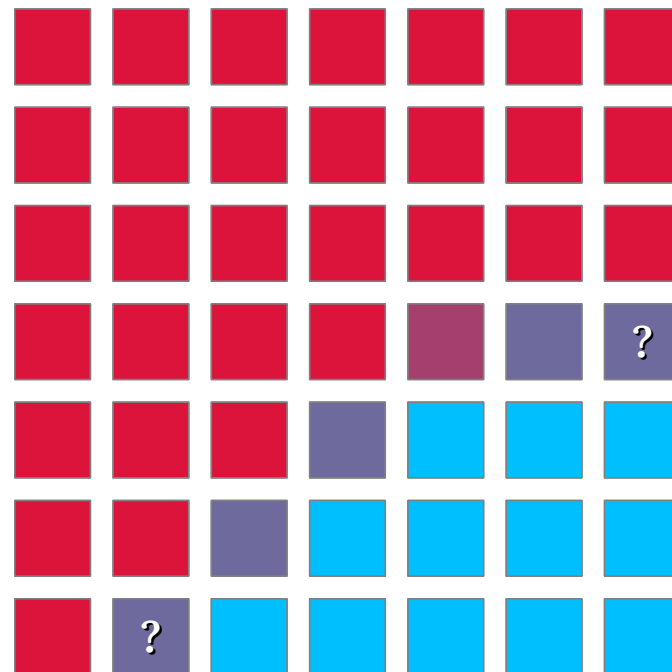
- Nearest-neighbor:
 - jagged, staircase-like edges (no sub-pixel precision)
- Bilinear / bicubic:
 - isotropic smoothing: interpolate across an edge the same as along it
 - result: visible edge blurring, loss of sharp detail
- These artifacts motivate interpolation methods that are aware of local image structure



- Idea: interpolate along the edge direction, not across it, using local gradient information
- Two-step interpolation:
 - step 1: interpolate missing pixels along the diagonal first (4 neighbours)
 - step 2: use a local consistency check, e.g. compare $|a-c|$ vs $|b-d|$, to decide which direction better preserves the edge



- Same approach for horizontal & vertical directions
- Reference: Z. Li, M. Orchard, "New Edge-Directed Interpolation" (NEDI), 2001

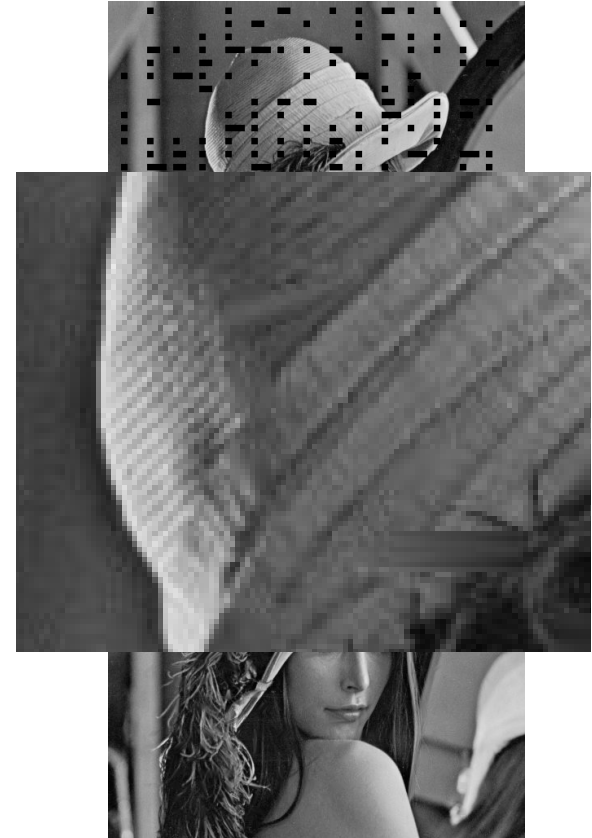


Application 1: Digital Zooming

- Classic use case: resolution enhancement for display or further processing
- Single-image methods: the interpolation techniques seen so far (bilinear, bicubic, edge-directed)
- Multi-frame super-resolution: combine several low-resolution frames of the same scene to recover extra detail
- Modern deep-learning super-resolution (CNNs, GANs) pushes this idea much further

Application 2: Image Inpainting/Error Concealment

- Goal: reconstruct missing or damaged image regions from the surrounding valid pixels
- Typical causes of missing data:
 - transmission errors (error concealment)
 - sensor defects, occlusions, user-requested object removal
- Classic approaches extend simple interpolation; modern approaches use texture synthesis or deep generative models



Application 2: Image Inpainting/Error Concealment

REMOVE ANY UNWANTED ELEMENTS



WATERMARK ↓



DATE STAMP ↓



EXTRA OBJECTS ↓



Application 3: Geometric Transformations

- A geometric transformation maps $(x,y) \rightarrow (x',y')$
- In practice we use the inverse mapping: for each output pixel (x',y') , compute the corresponding source position (x,y)
- (x,y) generally falls between pixels (non-integer coordinates)
- Solution: interpolate the source image at (x,y) using the techniques seen so far (e.g. `cv::remap` with `INTER_LINEAR`)

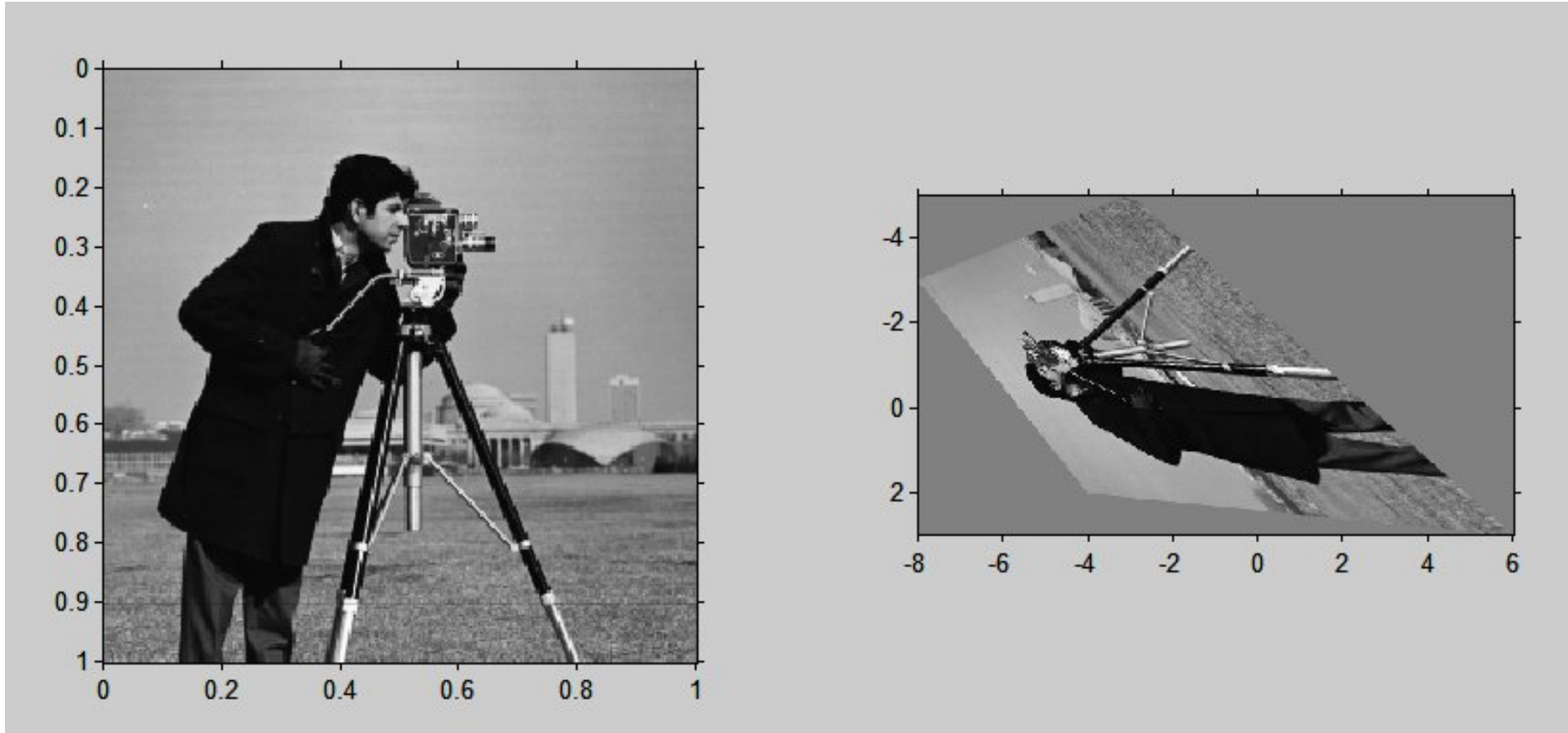
Application 3: Geometric Transformations



- Rotation (angle θ):
 - $[x'; y'] = [\cos\theta, \sin\theta; -\sin\theta, \cos\theta] \cdot [x; y]$
- Scaling (factor a):
 - $[x'; y'] = [a, 0; 0, 1/a] \cdot [x; y]$
- Shear:
 - $[x'; y'] = [1, 0; s, 1] \cdot [x; y] + [dx; dy]$
- Affine (6 DOF, maps a square to a parallelogram):
 - $[x'; y'] = [a_{11}, a_{12}; a_{21}, a_{22}] \cdot [x; y] + [dx; dy]$

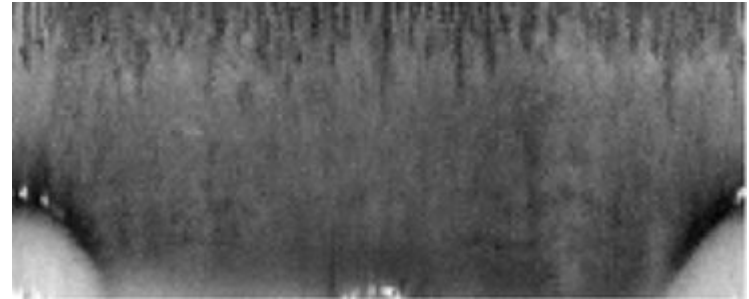
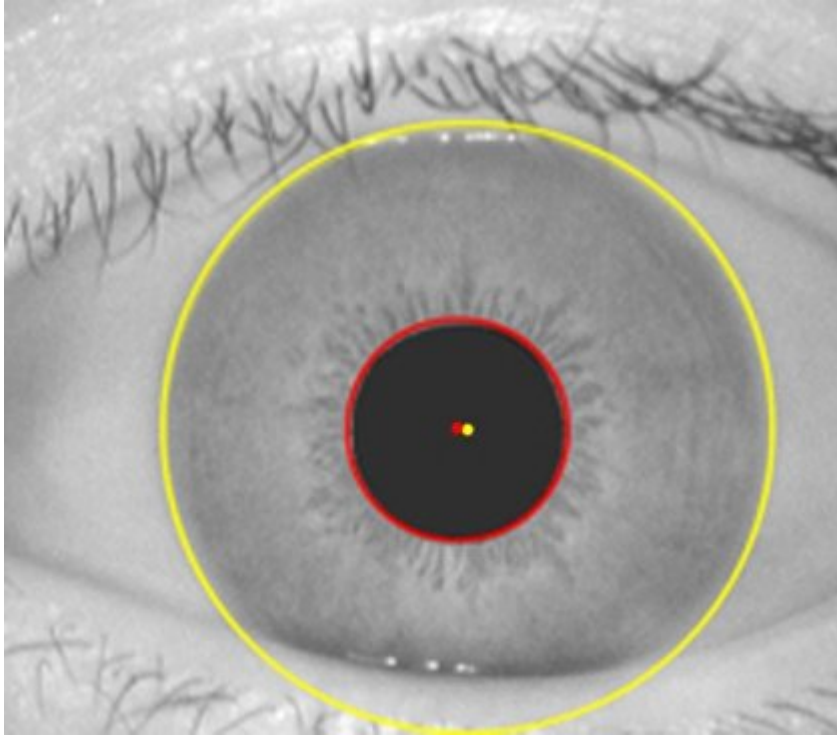
- 8 degrees of freedom: maps a square to a general quadrilateral (parallelism is not preserved)
- $x' = (a_1 \cdot x + a_2 \cdot y + a_3) / (a_7 \cdot x + a_8 \cdot y + 1)$
- $y' = (a_4 \cdot x + a_5 \cdot y + a_6) / (a_7 \cdot x + a_8 \cdot y + 1)$
- This is the same homography H we are going to introduce for planar scenes in the single-view and epipolar geometry lectures

Projective Transformations (Homography)



- Cartesian $\rightarrow$ Polar:
 - $r = \sqrt{x^2 + y^2}$
 - $\theta = \text{atan2}(y, x)$
- Polar $\rightarrow$ Cartesian:
 - $x = r \cdot \cos(\theta)$
 - $y = r \cdot \sin(\theta)$
- Applications:
 - iris image unwrapping for biometric recognition
 - creative/fisheye-like warps, e.g. $r \rightarrow \sqrt{r}$

Polar and Other Coordinate Transforms



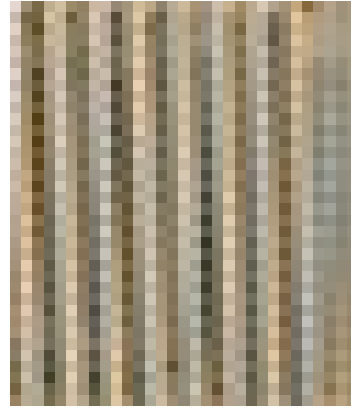
Application 4: Color Demosaicing

- We already saw Color Filter Array (CFA): each physical pixel senses only one color channel (Bayer pattern)
- Demosaicing = interpolating the two missing color channels at every pixel from neighboring same-color samples
- Approaches:
 - advanced, edge-aware CFA interpolation: fewer artifacts, higher computational cost

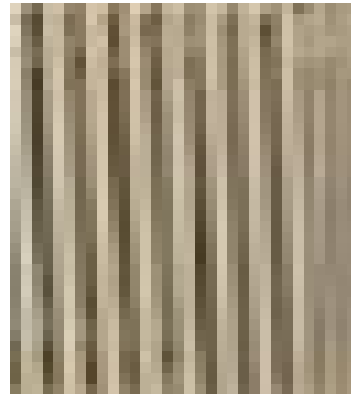
Application 4: Color Demosaicing



“Simple” interpolation



Edge-aware interpolation



- `cv::resize(src, dst, size, 0, 0, flag):`
 - `INTER_NEAREST`, `INTER_LINEAR`, `INTER_CUBIC`,
`INTER_LANCZOS4`
- `cv::remap(src, dst, map1, map2, interpolation):` general geometric warps via inverse mapping
- `cv::warpAffine` / `cv::warpPerspective`: ready-made wrappers around `remap` for affine / projective transforms

- Image interpolation bridges the discrete (sampled) and continuous world — a practical reconstruction problem
- Classic 1D/2D methods: nearest-neighbor, bilinear, bicubic — increasing quality and cost
- Linear methods have a structural weakness at edges; edge-directed methods address it
- A unifying tool used across many applications:
 - digital zooming / super-resolution
 - inpainting and error concealment
 - geometric transformations (rotation, affine, projective, polar)
 - color demosaicing
- Remains an active topic: modern deep-learning methods extend classic interpolation ideas